Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет
Программной Инженерии и Компьютерной Техники
Направление подготовки (специальность)
09.03.04 Программная инженерия
Дисциплина
Системы искусственного интеллекта

ЛАБОРАТОРНАЯ РАБОТА 1
ОТЧЕТ

Выполнил студент:
Быков Лев (489889)
Группа:
P3319
Преподаватель:
Болдырева Елена Александровна (157150)

г. Санкт-Петербург
2026

Содержание

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ **3**
ВВЕДЕНИЕ **4**
1 АНАЛИЗ ТРЕБОВАНИЙ **5**
1.1 Требования к системе поддержки принятия решений 5
1.2 Требования к базе знаний (Prolog) 5
1.3 Требования к онтологии (Protege) 5
2 ИЗУЧЕНИЕ ОСНОВНЫХ КОНЦЕПЦИЙ И ИНСТРУМЕНТОВ **6**
2.1 Базы знаний и онтологии 6
2.2 Prolog 6
2.3 Инструменты 6
3 РЕАЛИЗАЦИЯ СИСТЕМЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА **7**
3.1 База знаний на Prolog 7
3.2 Онтология в Protege 7
3.3 Тестирование и отладка 11
4 ОЦЕНКА И ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТОВ **13**
4.1 Примеры запросов к БЗ и онтологии 13
4.2 Соответствие требованиям 19
4.3 Интерпретация и возможности развития 19
ЗАКЛЮЧЕНИЕ **20**

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Часть 1. Создать базу знаний на языке Prolog на тему, связанную с играми (выбрана тема «Видеоигры»). База знаний должна содержать не менее 20 фактов с одним аргументом, 10–15 фактов с двумя аргументами, дополняющих и связывающих другие факты, и 5–7 правил. Написать запросы разной сложности: простые запросы к фактам, запросы с логическими операторами И, ИЛИ, НЕ, запросы с переменными и запросы, требующие выполнения правил. Снабдить код комментариями.

Часть 2. Перевести базу знаний в онтологию в среде Protege: преобразовать факты и отношения Prolog в классы, свойства и индивиды, построить иерархию классов, протестировать онтологию, продемонстрировать её визуализацию и выполнение запросов.

ВВЕДЕНИЕ

Цель работы — получить практические навыки представления знаний двумя способами: в виде базы знаний (БЗ) на языке логического программирования Prolog и в виде онтологии на языке OWL в среде Protege, а также сравнить эти подходы.

В качестве предметной области выбраны **видеоигры**: сами игры, их жанры, студии-разработчики и характеристики (наличие многопользовательского режима, открытого мира). На основе этих знаний система отвечает на вопросы, полезные игроку при выборе игры: «во что поиграть одному?», «во что поиграть с друзьями?», «какие RPG с открытым миром существуют?», «в каких жанрах работает студия?».

Значимость. Базы знаний и онтологии лежат в основе экспертных систем и систем поддержки принятия решений (СППР): знания хранятся отдельно от программы, а новые выводы получаются автоматически механизмом логического вывода. Даже на маленькой предметной области видны основные приёмы — факты, правила, иерархии понятий, вывод — и принципиальные различия между логическим программированием и онтологическим моделированием.

1 АНАЛИЗ ТРЕБОВАНИЙ

1.1 Требования к системе поддержки принятия решений

□3□a0Хранить сведения об объектах предметной области (играх, жанрах, разработчиках) и связях между ними.

□3□a0Отвечать на простые вопросы о фактах («является ли tetris игрой?»).

□3□a0Находить объекты по заданным характеристикам с помощью переменных.

□3□a0Поддерживать сложные условия с логическими операциями И, ИЛИ, НЕ.

□3□a0Выводить новые знания по правилам (рекомендации: одиночные игры, игры для компании и т.п.).

1.2 Требования к базе знаний (Prolog)

Соответствие реализованной базы знаний требованиям задания приведено в таблице 1.

Таблица 1 – Соответствие базы знаний требованиям задания

Требование задания	**Реализовано**
Не менее 20 фактов с одним аргументом	25 фактов: `game/1` (7), `genre/1` (5), `developer/1` (6), `multiplayer/1` (4), `open_world/1` (3)
10–15 фактов с двумя аргументами, связанных с другими фактами	14 фактов: `has_genre/2` (7), `developed_by/2` (7)
5–7 правил	6 правил: `single_player`, `open_world_rpg`, `good_for_friends`, `developer_genre`, `same_developer`, `multi_game_studio`
Запросы разной сложности (простые, с переменными, с И/ИЛИ/НЕ, через правила)	Подраздел 4.1
Комментарии к фактам, предикатам и правилам	Каждый предикат и правило в `games.pl` прокомментированы

1.3 Требования к онтологии (Protege)

□3□a0Каждому одноаргументному предикату-«типу» соответствует класс, предикатам-«свойствам» — подклассы.

□3□a0Каждому двухаргументному предикату соответствует объектное свойство с областью определения (domain) и областью значений (range).

□3□a0Константы Prolog становятся индивидами, факты — утверждениями о них (class / property assertions).

□3□a0Правилам соответствуют определяемые (эквивалентные) классы и цепочки свойств, чтобы ризонер выводил те же результаты.

□3□a0Онтология должна быть непротиворечивой, иметь иерархию классов, визуализироваться и отвечать на запросы.

2 ИЗУЧЕНИЕ ОСНОВНЫХ КОНЦЕПЦИЙ И ИНСТРУМЕНТОВ

2.1 Базы знаний и онтологии

База знаний — совокупность фактов и правил о предметной области, к которой применяется механизм логического вывода. Факты описывают то, что известно непосредственно, правила — как из известного получить новое.

Онтология — формальная явная спецификация понятийной модели предметной области: классы (понятия), их иерархия, свойства (отношения) и индивиды (экземпляры). Стандартный язык онтологий — OWL 2 (W3C), основанный на дескрипционных логиках. Программа-ризонер (HermiT, Pellet, ELK) проверяет онтологию на непротиворечивость, строит выведенную иерархию классов и определяет, к каким классам относятся индивиды.

Важное различие: Prolog работает в **предположении замкнутого мира** (всё, что не выводится, ложно), а OWL — в **предположении открытого мира** (то, что не сказано, просто неизвестно). Кроме того, в OWL нет предположения об уникальности имён: разные имена могут обозначать один объект, пока явно не сказано обратное.

2.2 Prolog

Prolog — декларативный язык логического программирования. Программа состоит из фактов (`game(tetris).`), правил (`Голова :- Тело.`) и запросов (`?- Цель.`). Для ответа на запрос интерпретатор использует унификацию (сопоставление с образцом) и поиск с возвратом (backtracking), перебирая все способы доказать цель. Логические связки: «,» — И, «;» — ИЛИ, «\+» — НЕ (отрицание как неудача). Благодаря этому Prolog удобен для экспертных систем, обработки естественного языка, планирования и других задач ИИ.

2.3 Инструменты

□3□a0**SWI-Prolog** — свободная реализация Prolog с интерактивной консолью; в нём выполнялись все запросы к БЗ. Содержит библиотеку `semweb` для работы с RDF/OWL из Prolog.

□3□a0**Protege 5.6.9** — редактор онтологий от Стэнфордского университета: редактирование классов, свойств и индивидов, встроенные ризонеры HermiT и ELK, вкладка DL Query для запросов на манчестерском синтаксисе, SPARQL Query, визуализация OntoGraf и OWLViz.

□3□a0**HermiT** — ризонер для OWL 2 DL, использовался для вывода и проверки непротиворечивости.

3 РЕАЛИЗАЦИЯ СИСТЕМЫ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

3.1 База знаний на Prolog

Файл `games.pl` содержит 7 игр, 5 жанров, 6 разработчиков, сведения о многопользовательском режиме и открытом мире, а также связи «игра – жанр» и «игра – разработчик». Фрагмент фактов:

```
game(witcher3).             genre(rpg).          developer(cd_projekt_red).
multiplayer(minecraft).     open_world(witcher3).
has_genre(witcher3, rpg).   developed_by(witcher3, cd_projekt_red).
```

Правила логического вывода, на основе которых система «принимает решения» (рекомендует игры и делает выводы о студиях):

```
% НЕ: игра без многопользовательского режима
single_player(G) :- game(G), \+ multiplayer(G).

% И: RPG с открытым миром
open_world_rpg(G) :- has_genre(G, rpg), open_world(G).

% И + ИЛИ: игра для компании друзей
good_for_friends(G) :- multiplayer(G),
    ( has_genre(G, sports) ; has_genre(G, shooter) ; has_genre(G, sandbox)
).

% в каких жанрах работает студия (соединение двух отношений)
developer_genre(D, Genre) :- developed_by(G, D), has_genre(G, Genre).

% две разные игры одной студии
same_developer(G1, G2) :- developed_by(G1, D), developed_by(G2, D), G1 \=
G2.

% студия, выпустившая больше одной игры (использует правило same_developer)
multi_game_studio(D) :- same_developer(G1, G2), G1 @< G2, developed_by(G1,
D).
```

3.2 Онтология в Protege

Онтология `games.owl` (IRI `http://www.semanticweb.org/lab1/games`) построена прямым переводом БЗ. Соответствие элементов приведено в таблице 2.

Таблица 2 – Соответствие элементов базы знаний Prolog и онтологии OWL

Prolog	**OWL**
`game/1`, `genre/1`, `developer/1`	Классы `Game`, `Genre`, `Developer` (попарно непересекающиеся)
`multiplayer/1`, `open_world/1`	Подклассы `MultiplayerGame`, `OpenWorldGame` класса `Game`
Константы (`witcher3`, `rpg`, …)	Индивиды (18 шт.), объявлены попарно различными (AllDifferent)
`has_genre/2`	Объектное свойство `hasGenre`: Game → Genre
`developed_by/2`	`developedBy`: Game → Developer и обратное `developerOf`
`single_player`	`SinglePlayerGame ≡ Game and not MultiplayerGame`
`open_world_rpg`	`OpenWorldRPG ≡ OpenWorldGame and (hasGenre value rpg)`
`good_for_friends`	`GameForFriends ≡ MultiplayerGame and (hasGenre some {sports, shooter, sandbox})`
`developer_genre`	Свойство `worksInGenre` с цепочкой `developerOf o hasGenre`
`same_developer`	Свойство `sameDeveloperAs` с цепочкой `developedBy o developerOf`
`multi_game_studio`	`MultiGameStudio ≡ Developer and (developerOf min 2 Game)`

Чтобы отрицание в `SinglePlayerGame` работало в открытом мире OWL, перечень многопользовательских игр закрыт явно: `MultiplayerGame ≡ {minecraft, dark_souls, fifa23, doom}`. Аксиома AllDifferent нужна, чтобы ризонер мог считать различные игры (ограничение «min 2») и убедиться, что, например, `witcher3` не совпадает ни с одной из многопользовательских игр. У каждого класса и свойства есть аннотация `rdfs:comment` с указанием соответствующего предиката или правила Prolog.

Иерархия классов после работы ризонера HermiT показана на рисунке 1 (пунктиром — связи, выведенные ризонером из определений).

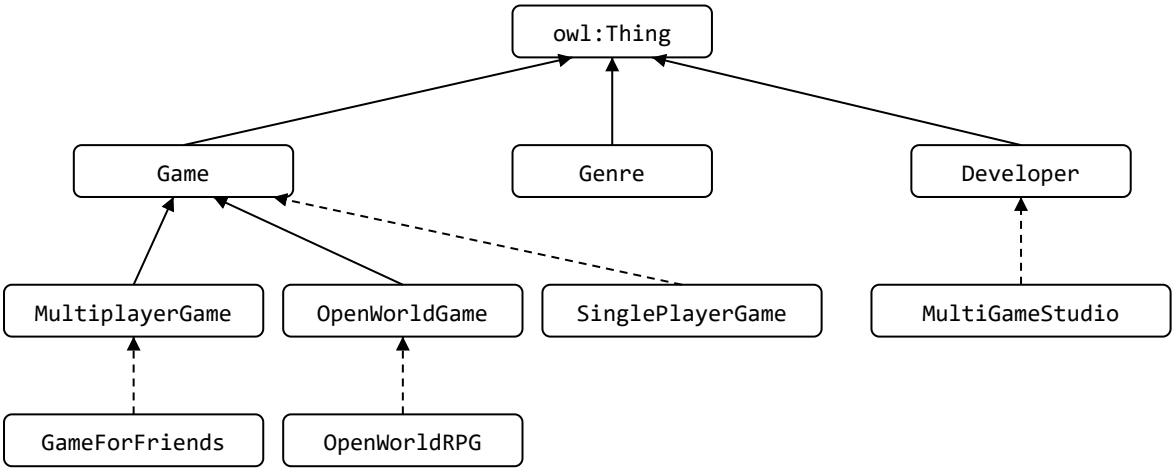


Рисунок 1 – Иерархия классов онтологии (стрелка: «является подклассом»)

Онтология в среде Protege показана на рисунках 2–6: метрики онтологии, выведенная ризонером иерархия классов (у определяемых классов в разделе Equivalent To

видно определение, а выведенные факты подсвечены жёлтым), объектные свойства с цепочкой свойств и индивиды с выведенными типами и связями.

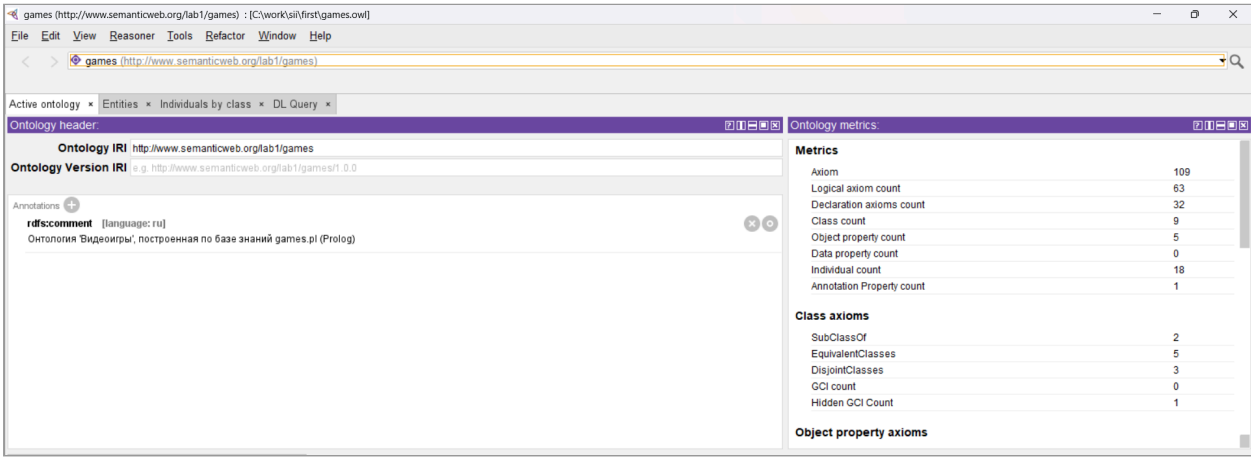


Рисунок 2 – Вкладка Active ontology: метрики онтологии (9 классов, 5 объектных свойств, 18 индивидов)

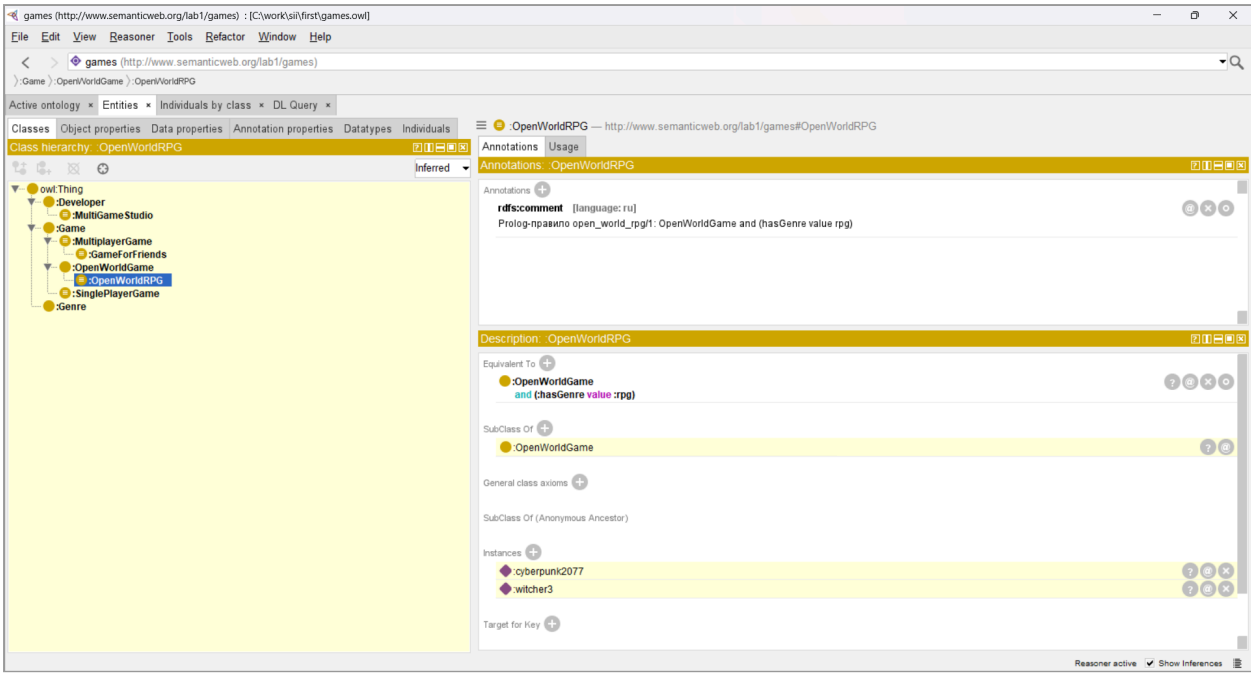


Рисунок 3 – Выведенная иерархия классов (Inferred) и определение класса OpenWorldRPG

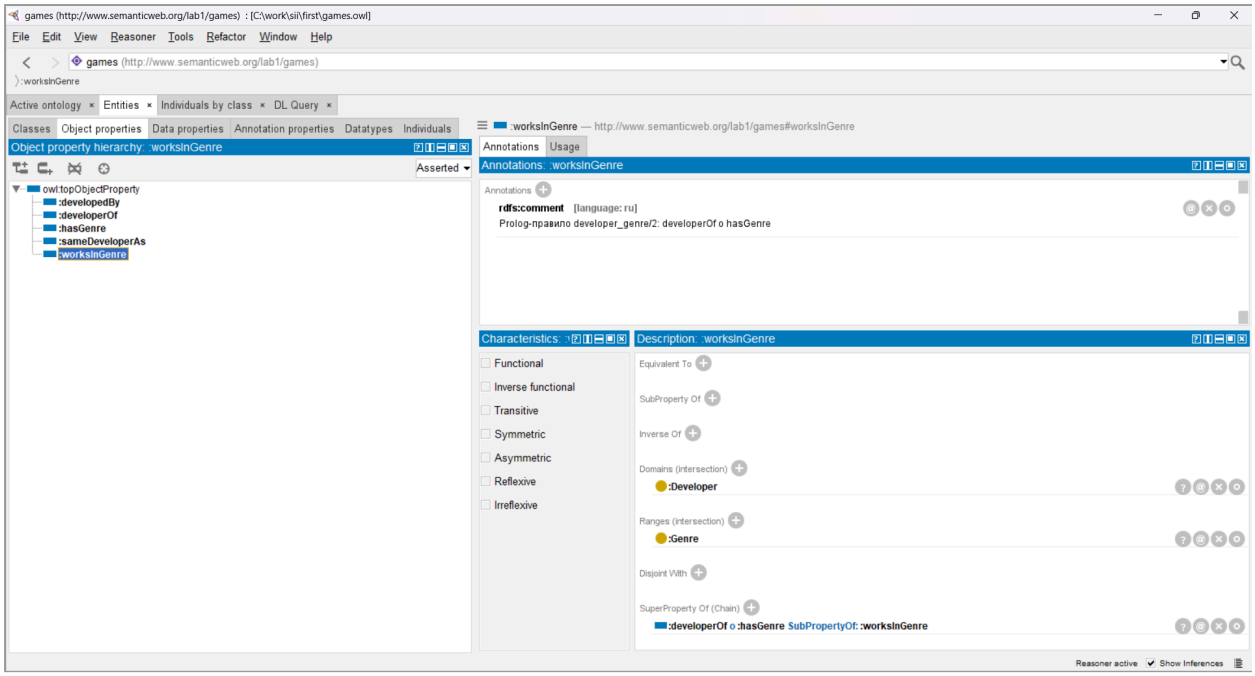


Рисунок 4 – Объектные свойства; свойство worksInGenre задано цепочкой developerOf o hasGenre

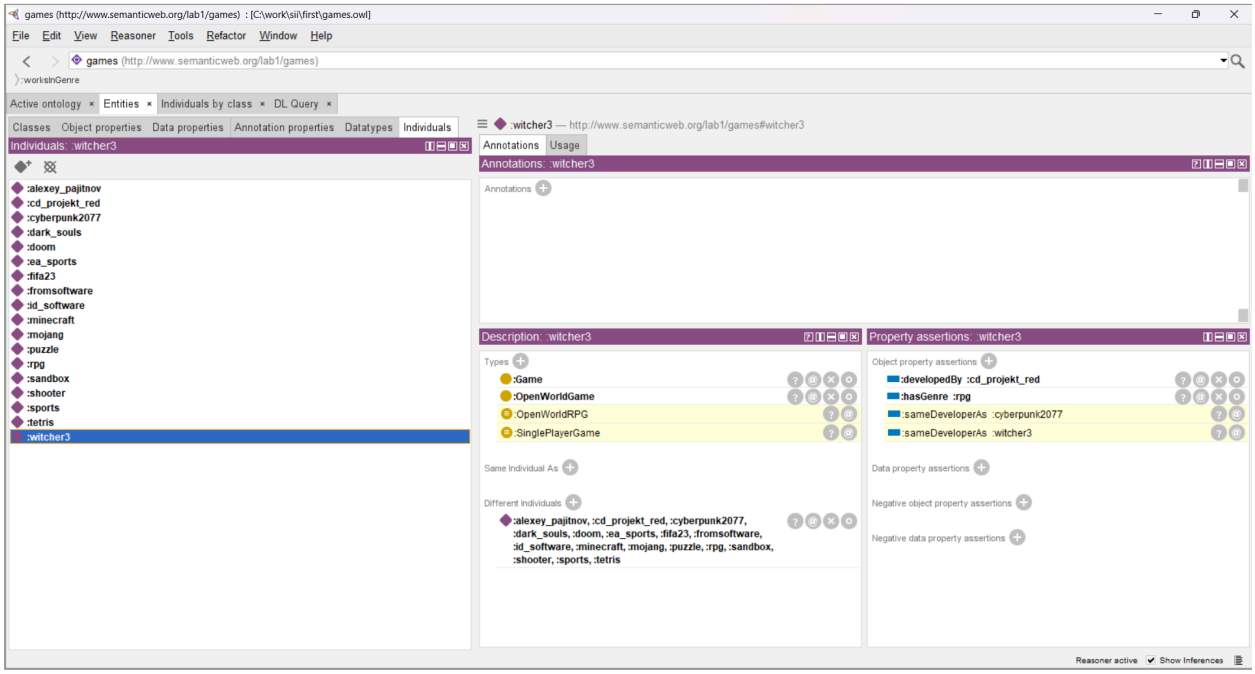


Рисунок 5 – Индивид witcher3: утверждённые и выведенные ризонером (жёлтые) типы и свойства

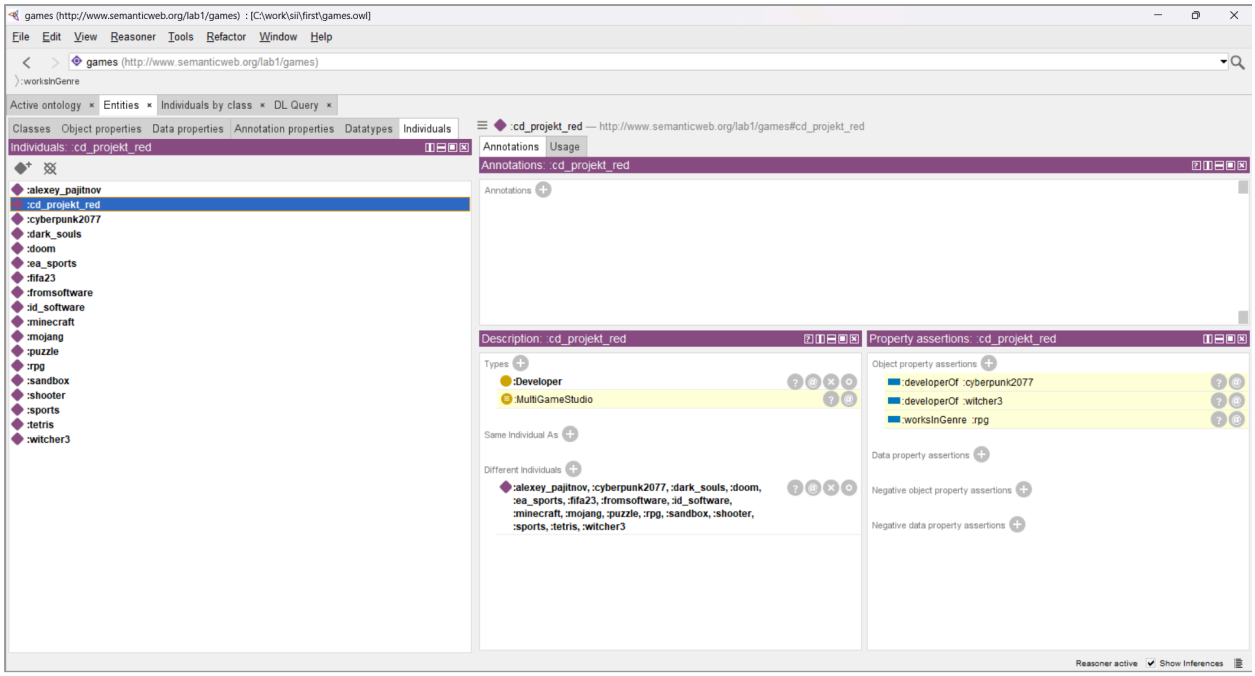


Рисунок 6 – Индивид cd_projekt_red: выведены тип MultiGameStudio и свойство worksInGenre rpg

3.3 Тестирование и отладка

БЗ загружалась в SWI-Prolog 10 командой `swipl games.pl`; все запросы из подраздела 4.1 выполнены, ответы совпали с ожидаемыми. При отладке выявлены и учтены особенности: в правиле `single_player` переменная связывается через `game(G)` до отрицания (иначе `\+` с несвязанной переменной даёт неверный результат); в `multi_game_studio` добавлено условие `G1 @< G2`, чтобы одна студия не выдавалась дважды из-за симметричных пар.

Онтология открыта в Protege 5.6.9, запущен ризонер HermiT (Reasoner → HermiT → Start reasoner). Онтология непротиворечива (109 аксиом), выведенная иерархия совпала с ожидаемой (рис. 1), индивиды правильно классифицированы. Визуализация выполнена на вкладке OntoGraf (рисунки 7, 8), запросы — на вкладке DL Query (подраздел 4.1).

На графе OntoGraf узлы-кружки — классы, ромбы — индивиды. Цвет рёбер обозначает тип связи: сплошные фиолетовые — «индивид принадлежит классу», синие — «подкласс», оранжевые пунктирные — свойство hasGenre, жёлтые пунктирные — свойство developedBy.

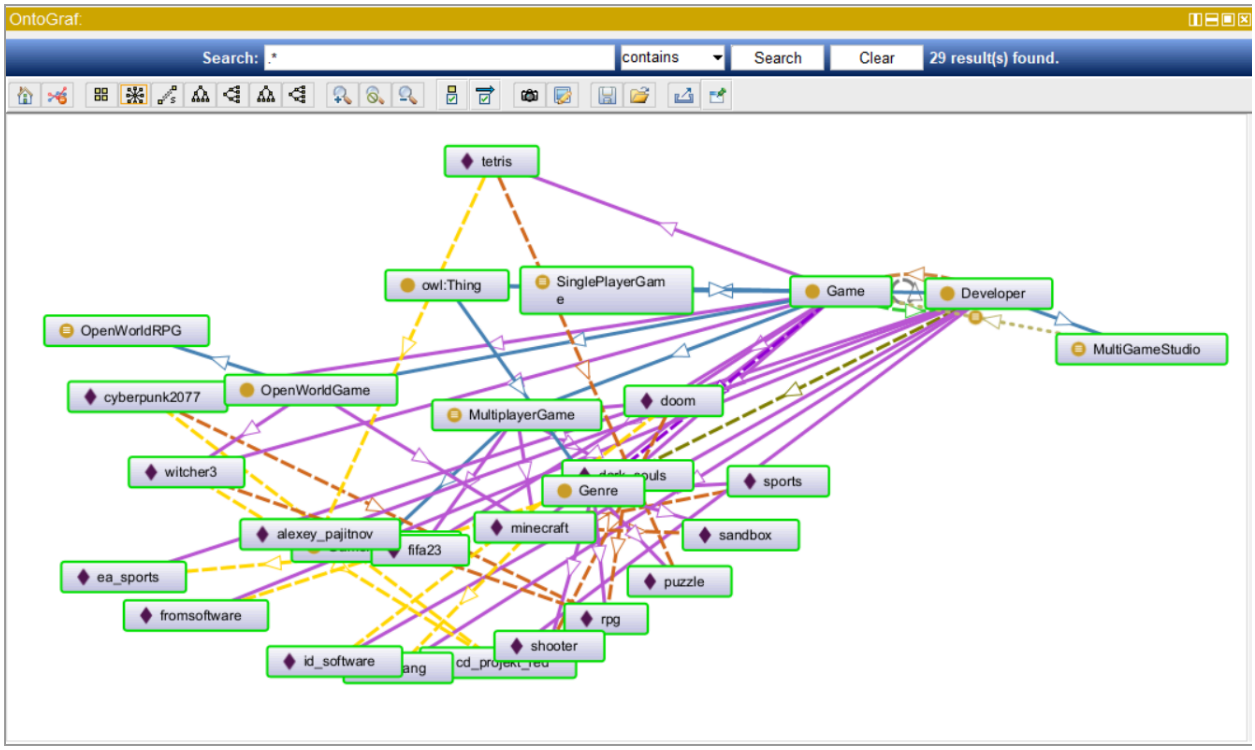


Рисунок 7 – Граф всей онтологии в OntoGraf (все классы, индивиды и связи между ними)

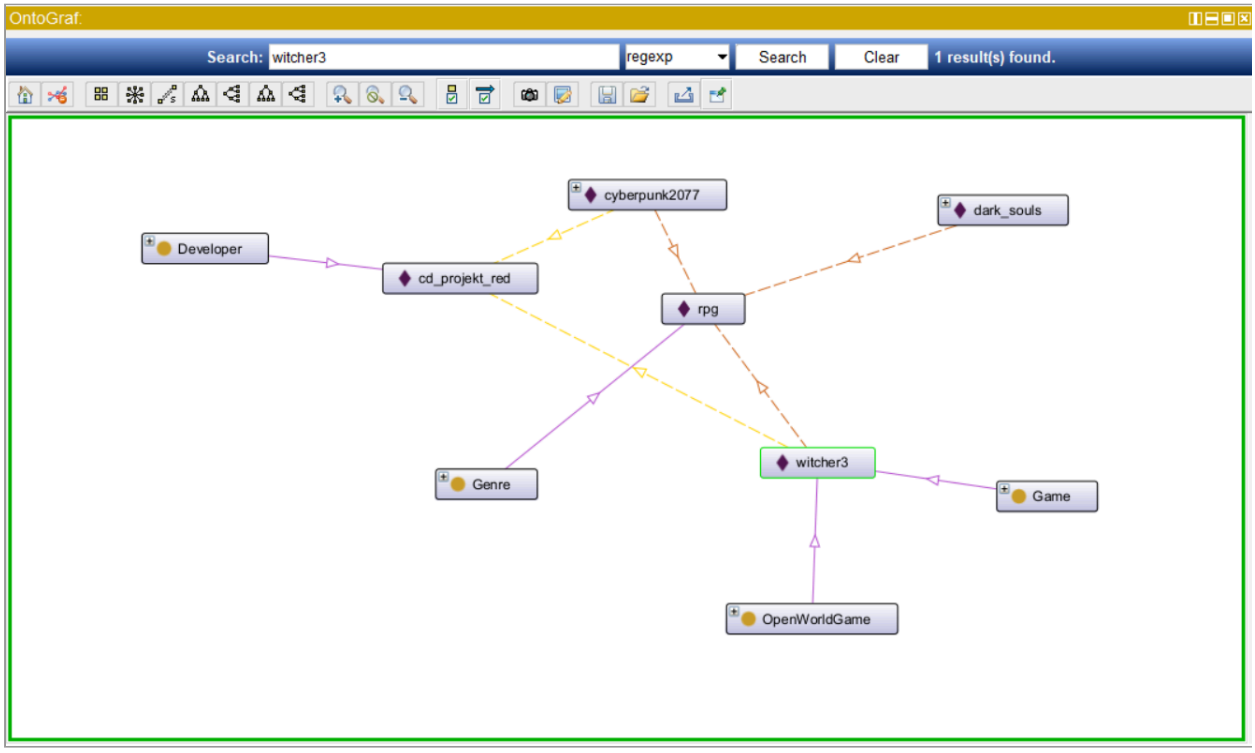


Рисунок 8 – Граф связей игры witcher3: жанр rpg, студия cd_projekt_red, классы; раскрыты соседи cd_projekt_red и rpg

4 ОЦЕНКА И ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТОВ

4.1 Примеры запросов к БЗ и онтологии

Результаты выполнения запросов к базе знаний в SWI-Prolog приведены на рисунках 9–12. Нажатие «;» запрашивает следующий ответ, «false» означает, что других ответов нет.

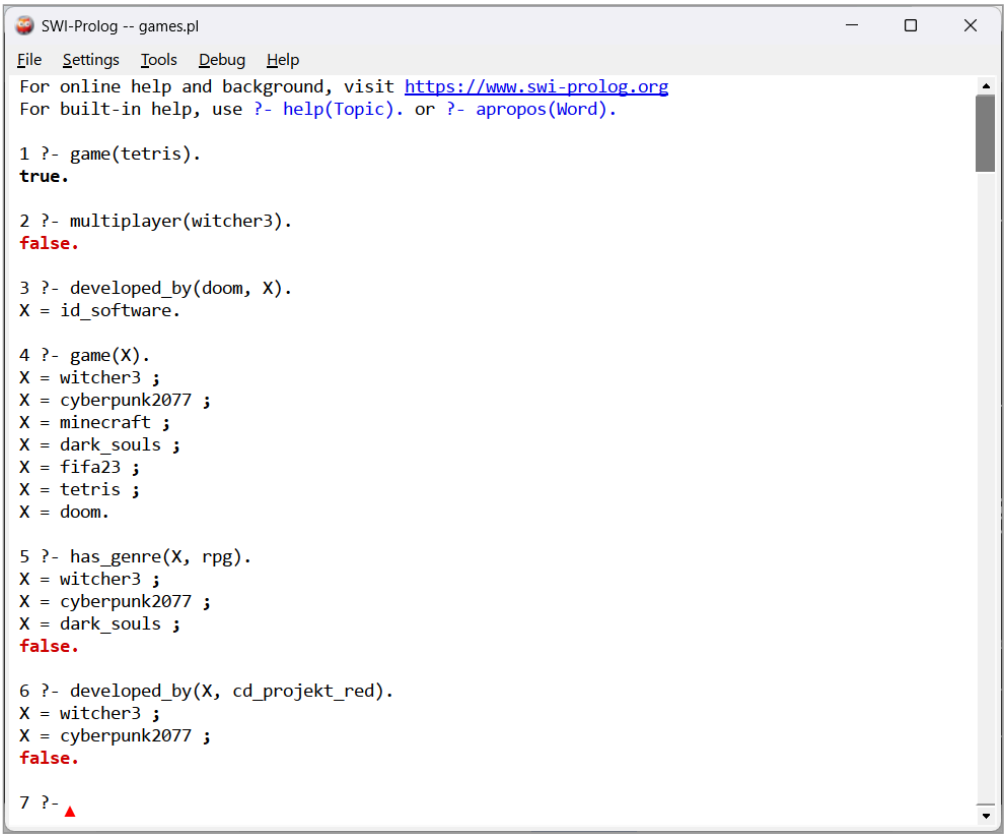


Рисунок 9 – Простые запросы к фактам и запросы с переменными

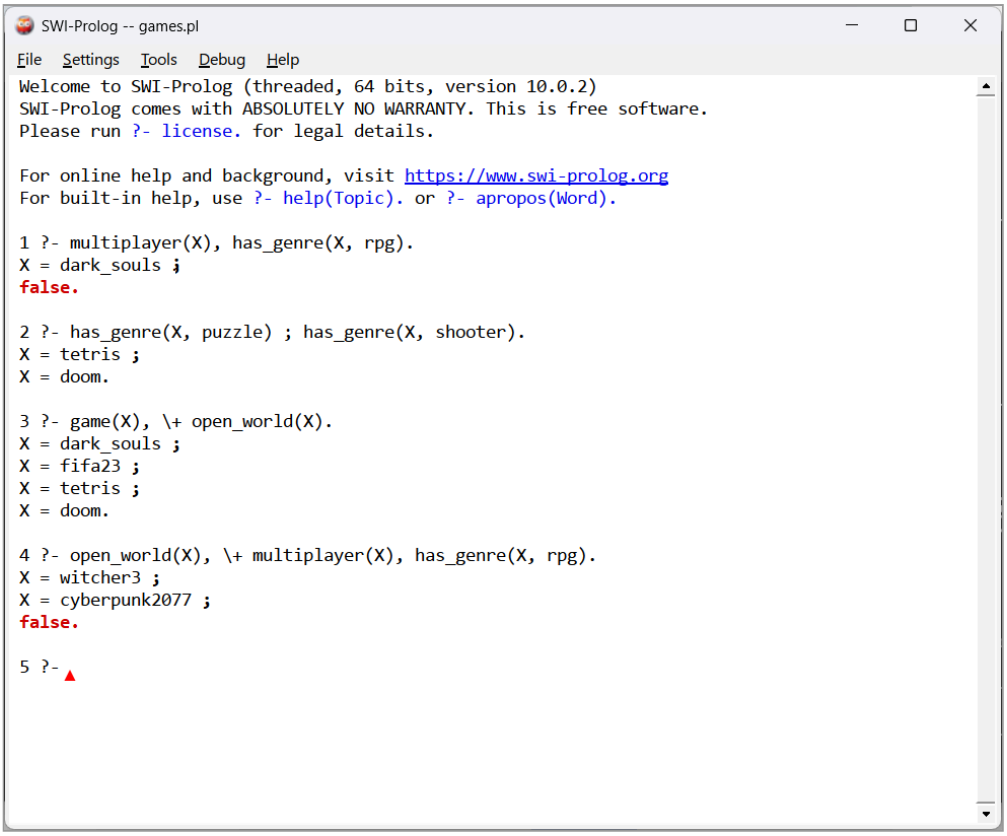


Рисунок 10 – Запросы с логическими операторами И, ИЛИ, НЕ

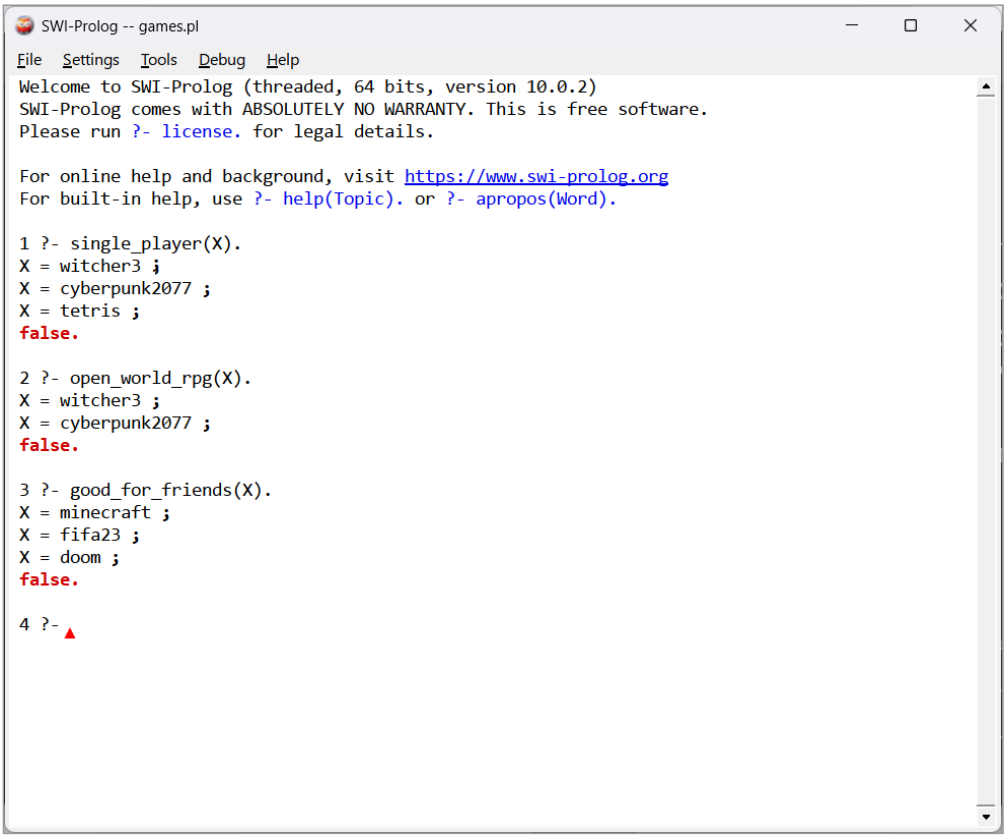


Рисунок 11 – Запросы, использующие правила single_player, open_world_rpg, good_for_friends

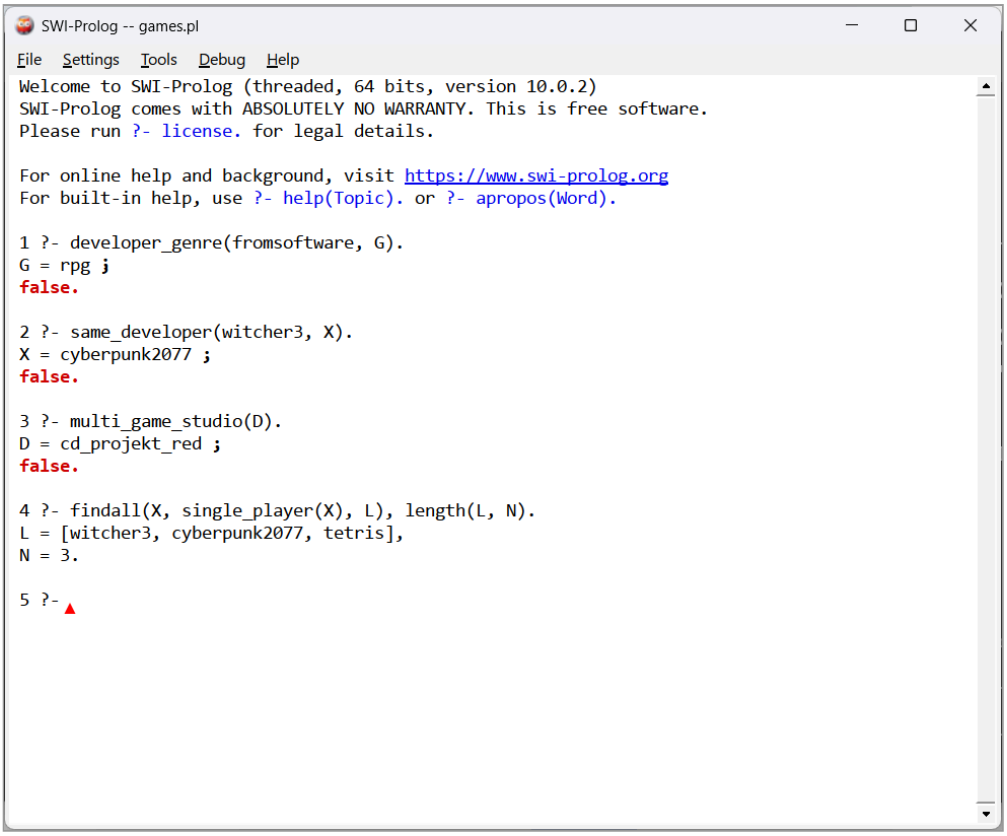


Рисунок 12 – Запросы, использующие правила developer_genre, same_developer, multi_game_studio, и запрос с findall

Результаты аналогичных запросов к онтологии на вкладке DL Query (с отметкой Instances, при запущенном ризонере HermiT) приведены на рисунках 13–18.

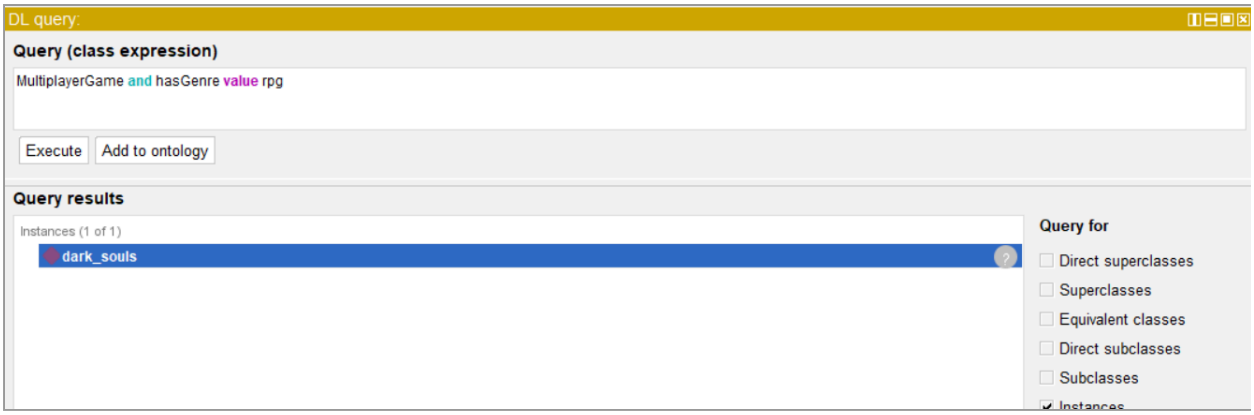


Рисунок 13 – DL Query: MultiplayerGame and hasGenre value rpg (логическое И)

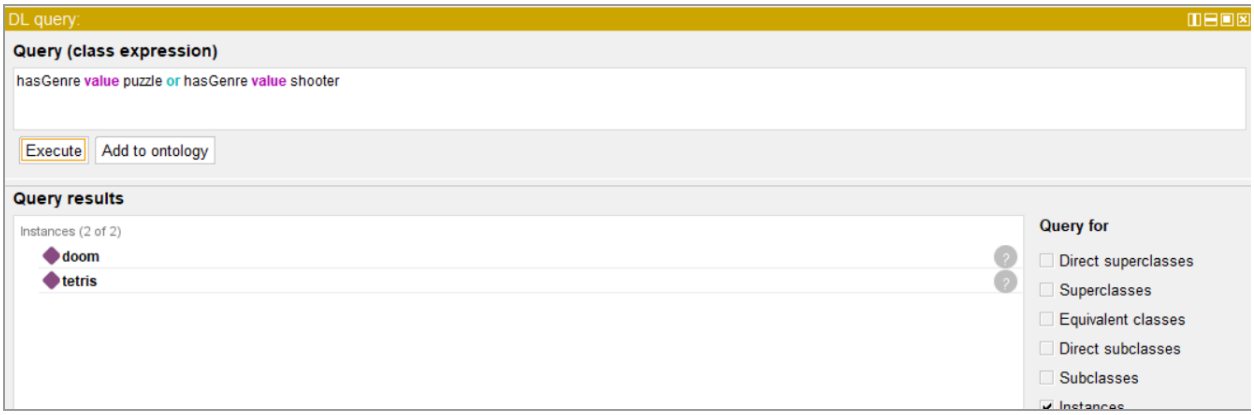


Рисунок 14 – DL Query: hasGenre value puzzle or hasGenre value shooter (логическое ИЛИ)

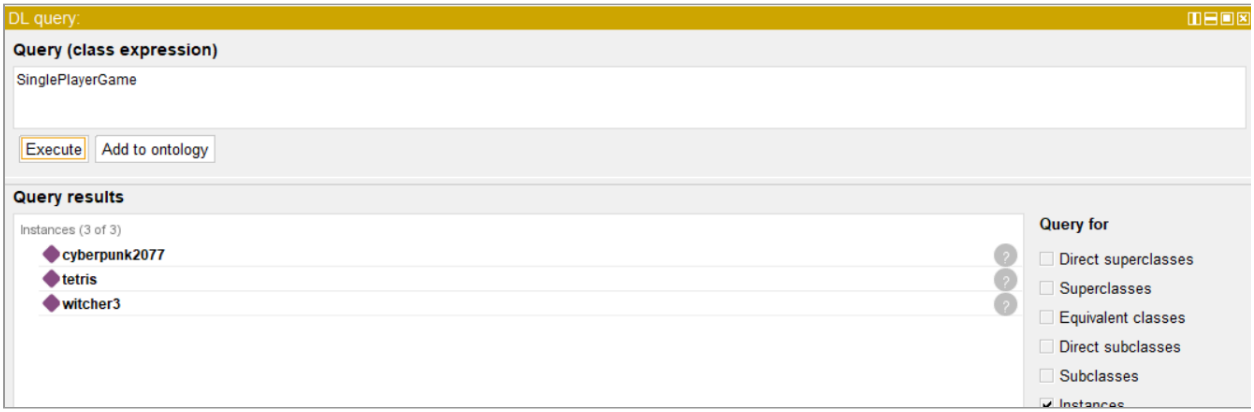


Рисунок 15 – DL Query: SinglePlayerGame (аналог правила single_player с отрицанием)

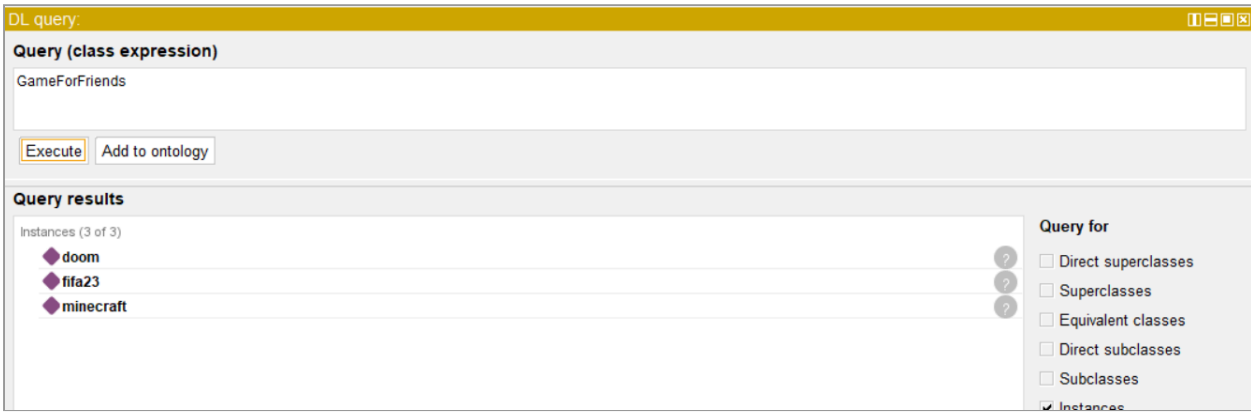


Рисунок 16 – DL Query: GameForFriends (аналог правила good_for_friends)

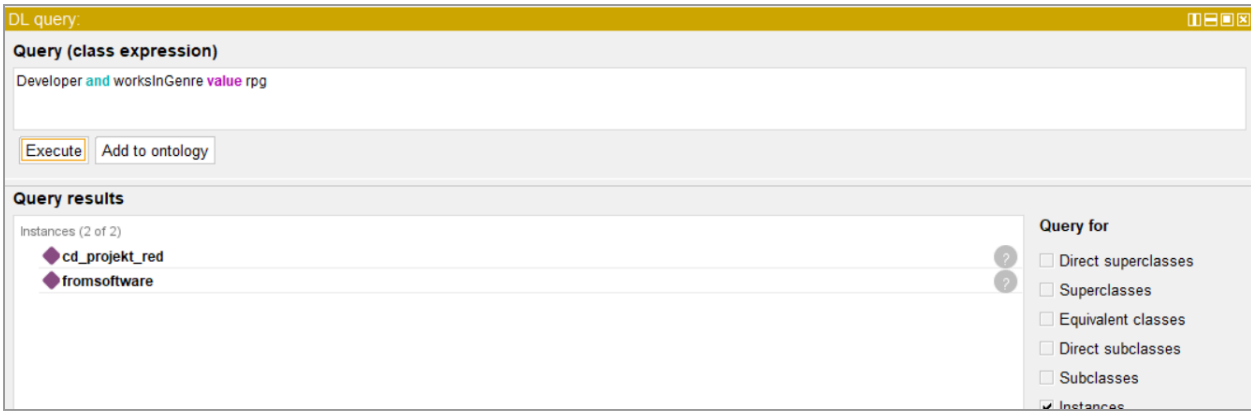


Рисунок 17 – DL Query: Developer and worksInGenre value rpg (аналог правила developer_genre)

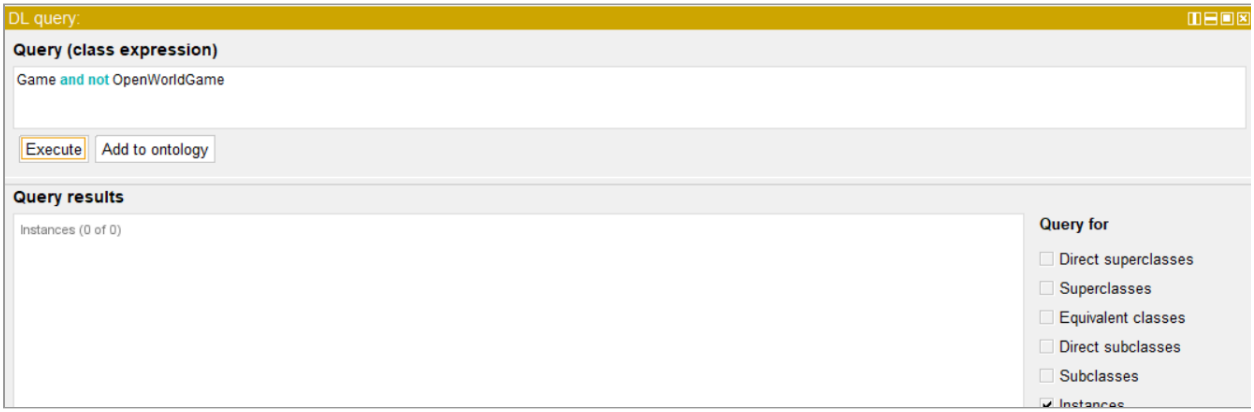


Рисунок 18 – DL Query: Game and not OpenWorldGame — пустой результат из-за предположения открытого мира

Сравнение ответов БЗ и онтологии на аналогичные запросы приведено в таблице 3.

Таблица 3 – Сравнение запросов к базе знаний и к онтологии

Тип	**Запрос Prolog**	**DL Query (Protege)**	**Результат**
Простой	`game(tetris).`	— (видно в Types индивида)	true
Простой	`developed_by(doom, X).`	`developerOf value doom`	id_software
Переменные	`has_genre(X, rpg).`	`hasGenre value rpg`	witcher3, cyberpunk2077, dark_souls
Переменные	`developed_by(X, cd_projekt_red).`	`developedBy value cd_projekt_red`	witcher3, cyberpunk2077
И	`multiplayer(X), has_genre(X, rpg).`	`MultiplayerGame and hasGenre value rpg`	dark_souls
ИЛИ	`has_genre(X,puzzle) ; has_genre(X,shooter).`	`hasGenre value puzzle or hasGenre value shooter`	tetris, doom
НЕ (правило)	`single_player(X).`	`SinglePlayerGame`	witcher3, cyberpunk2077, tetris
Правило	`open_world_rpg(X).`	`OpenWorldRPG`	witcher3, cyberpunk2077
Правило (И+ИЛИ)	`good_for_friends(X).`	`GameForFriends`	minecraft, fifa23, doom
Правило	`developer_genre(D, rpg).`	`Developer and worksInGenre value rpg`	cd_projekt_red, fromsoftware
Правило	`same_developer(witcher3, X).`	`sameDeveloperAs value witcher3`	Prolog: cyberpunk2077; OWL: cyberpunk2077, witcher3
Правило над правилом	`multi_game_studio(D).`	`MultiGameStudio`	cd_projekt_red
НЕ (без закрытия)	`game(X), \+ open_world(X).`	`Game and not OpenWorldGame`	Prolog: dark_souls, fifa23, tetris, doom; OWL: пусто

Сравнение реализаций. На позитивных запросах (И, ИЛИ, переменные, правила без отрицания) обе системы дают одинаковые ответы. Различия проявляются в трёх местах:

1. **Отрицание.** Prolog считает `\+ open_world(dark_souls)` истинным, потому что факта нет (замкнутый мир). OWL на запрос `Game and not OpenWorldGame` отвечает пустым множеством: из отсутствия утверждения не следует его ложность (открытый мир). Для `SinglePlayerGame` это решено явным перечислением многопользовательских игр и аксиомой AllDifferent.
2. **Неравенство.** В Prolog правило `same_developer` содержит `G1 \= G2`; в цепочке свойств OWL такого условия выразить нельзя, поэтому игра оказывается «от той же студии», что и она сама.

3. **Повторы.** Prolog возвращает ответ на каждый вариант доказательства (`developer_genre(D, rpg)` выдаёт `cd_projekt_red` дважды), ризонер OWL возвращает множество индивидов без повторов.

Также в Prolog правило — это процедура, вызываемая при запросе, а в OWL определение класса — аксиома, по которой ризонер заранее классифицирует все индивиды и строит иерархию (например, сам выводит, что `OpenWorldRPG` — подкласс `OpenWorldGame`, а `GameForFriends` — подкласс `MultiplayerGame`).

4.2 Соответствие требованиям

Все требования задания выполнены: БЗ содержит 25 одноаргументных фактов, 14 двухаргументных и 6 правил, использующих операции И, ИЛИ, НЕ и вызов одного правила из другого; продемонстрированы простые запросы, запросы с переменными, составные запросы и запросы через правила; код прокомментирован. Онтология содержит иерархию из 9 классов, 5 объектных свойств и 18 индивидов, все предикаты и правила Prolog переведены в элементы OWL, онтология непротиворечива, визуализирована и протестирована запросами DL Query.

4.3 Интерпретация и возможности развития

Система корректно отвечает на вопросы выбора игры: одному пользователю рекомендуются witcher3, cyberpunk2077 и tetris, компании друзей — minecraft, fifa23 и doom. Возможные направления развития:

□3□a0добавить числовые атрибуты (год выхода, рейтинг, возрастное ограничение) — в Prolog как факты с числом, в OWL как свойства данных — и правила с порогами;

□3□a0добавить профили игроков с предпочтениями и правило подбора игры под конкретного игрока;

□3□a0использовать правила SWRL в Protege для выражения условий, которые не выражаются стандартными аксиомами OWL;

□3□a0загружать онтологию в Prolog через библиотеку `semweb` SWI-Prolog и объединить оба подхода в одной системе.

ЗАКЛЮЧЕНИЕ

В ходе работы создана база знаний о видеоиграх на Prolog и построенная по ней онтология в Protege. Prolog показал себя удобным инструментом для быстрого описания фактов и гибких правил вывода с отрицанием, неравенством и произвольными вычислениями; его преимущество — простота и выразительность правил. Онтология OWL даёт стандартизованное, переиспользуемое описание понятий с явной иерархией классов, автоматической проверкой непротиворечивости и классификацией объектов ризонером, а также наглядную визуализацию; её особенность — работа в открытом мире, которую нужно учитывать при моделировании отрицания.

Разработанная система может служить основой рекомендательного сервиса или экспертной системы выбора игр, каталога игровой библиотеки, а сам подход — для построения СППР в любой предметной области, где важно явно хранить знания и объяснимо получать из них выводы.